Bab IV Parameterized Test

4.1 Tujuan Pembelajaran

1. Mahasiswa mampu memahami tipe-tipe parameter dalam *test method*
2. Mahasiswa mampu mengimplementasikan tipe-tipe parameter pada pengujian

4.2 Dasar Teori

Parameterized test adalah fitur yang diperkenalkan pada JUnit5. Fitur ini memungkinkan kita untuk mengeksekusi test method beberapa kali dengan parameter yang berbeda-beda atau dengan berbagai nilai input. Hal ini akan sangat membantu ketika kita ingin melakukan pengujian terhadap berbagai kasus atau skenario tanpa harus membuat test method yang berbeda untuk setiap inputnya. Untuk menggunakan parameterized test, kita harus mengganti anotasi @Test dengan anotasi @ParameterizedTest

Berikut ini adalah beberapa cara mengimplementasikan parameterized test:

1. Anotasi @ValueSource

   Anotasi ini digunakan untuk mendefinisikan *single-value parameter* atau parameter yang hanya berisi satu nilai. Parameter yang didefinisikan bisa menggunakan berbagai tipe data. Berikut adalah syntax yang dapat digunakan:

```
@ParameterizedTest
@ValueSource(ints = {nilai1,nilai2,nilai3,...,nilai-n})
```

2. Anotasi @CsvSource

   Anotasi ini digunakan untuk memberikan parameter melalui format Csv (Comma-separated-value). Selain itu kita juga dapat menggunakan anotasi @CsvFileSource untuk menyediakan parameter dengan format Csv melalui file eksternal. Berikut adalah syntax yang dapat digunakan:

```
@ParameterizedTest
@CsvSource({"a1,b1","a2,b2","a3,b3",...})
```

3. Anotasi @MethodSource

   Parameter dapat kita sediakan juga melalui method lain. Method tersebut harus mengembalikan stream atau iterable objek sebagai sumber parameter. Anotasi ini digunakan untuk mendefinisikan nama method tersebut.

   Pertama kita harus membuat method sebagai penyedia parameter. Berikut adalah contoh methodnya:

```
private static Stream<Arguments> provideParameters() {
    return Stream.of(
            Arguments.of( ...arguments: "x1", "y1"),
            Arguments.of( ...arguments: "x2", "y2"),
            Arguments.of( ...arguments: "x3", "y3"),
            Arguments.of( ...arguments: "...", "...")
    );
}
```

Setelah itu kita dapat menggunakan method *provideParameters* tersebut pada *test method* dengan syntax sebagai berikut:

```
@ParameterizedTest
@MethodSource("provideParameters")
```

4.3 Latihan

Latihan pada modul ini menggunakan studi kasus kalkulator.

1. Tambahkan sebuah method pada kelas Kalkulator untuk mengecek apakah sebuah nilai adalah nilai Ganjil atau Genap. Kembalian dari method ini adalah Boolean.

```
public boolean cekEven(int number){return number%2==0; }
```

2. Buatlah sebuat test method untuk menguji method cekEven. Gunakan parameterized test dengan memanfaatkan ValueSource.

```
@ParameterizedTest(name = "Eksekusi ke- {index}, nilai: {arguments}")
@ValueSource(ints = {2,4,6})
void cekEven(int param) {
    Kalkulator kalk = new Kalkulator();
    assertTrue(kalk.cekEven(param));
}
```

3. Jalankan dan perhatikan hasilnya. Test method cekEven akan dijalankan 3 kali sesuai jumlah parameter yang didefinisikan.

cekEven(int)	107 ms
Eksekusi ke- 1, nilai: 2	102 ms
Eksekusi ke- 2, nilai: 4	3 ms
Eksekusi ke- 3, nilai: 6	2 ms

4. Gunakan parameterized test dengan CsvSource pada method tambah. Pada argumennya kita dapat mendefinisikan beberapa nilai yaitu nilai a, nilai b, dan expected resultnya.

```
@ParameterizedTest
@CsvSource({"2,4,6","1,2,3","100,500,600"})
void tambahParameterized(int a, int b, int expect) {
    Kalkulator kalk = new Kalkulator(a,b);
    int actual = kalk.tambah();
    assertEquals(expect,actual);
}
```

5. Gunakan parameterized test dengan MethodSource pada method tambah. Buatlah terlebih dahulu method untuk mengirimkan stream parameter.

```java
private static Stream<Arguments> provideParameters() {
    return Stream.of(
            Arguments.of( ...arguments: "2", "4","6"),
            Arguments.of( ...arguments: "1", "2","3"),
            Arguments.of( ...arguments: "100", "500","600")
    );
}
```

6. Buatlah test method dengan anotasi MethodSource dan mengambil nilai dari method provide Parameters.

```java
@ParameterizedTest
@MethodSource("provideParameters")
void tambahparam(int a, int b, int expect) {
    Kalkulator kalk = new Kalkulator(a,b);
    int actual = kalk.tambah();
    assertEquals(expect,actual);
}
```

7. Jalankan test method yang telah dibuat dan perhatikan hasilnya

4.4 Tugas

Implementasikan parameterized test pada studi kasus Wallet.

1. Gunakan *single-value parameter* untuk menguji nominal cash valid (angka positif) dan nominal cash tidak valid (angka negatif)
2. Buatlah file csv untuk 2 kasus valid dan invalid data (withdraw cash). Gunakan @CsvFileSource untuk mengakses parameter dari file.
   - Contoh data valid:

     deposit,withdraw,expectedTotal

     10000,0,10000

     20000,5000,15000

     50000,25000,25000

     100000,100000,0

- Contoh data tidak valid

  Saat ada pengurangan nilai yang tidak valid atau saldo kurang maka fungsi withdraw akan melempar exception. Lakukan assertion untuk memastikan fungsi withdraw melempar exeption yang sesuai.

  initialDeposit,withdraw,exceptionType

  10000,15000,InsufficientFundsException

  10000,0,IllegalArgumentException

  10000,-1000,IllegalArgumentException

  0,1000,InsufficientFundsException

3. Pada studi kasus sebelumnya **owner** bertipe *string*. Pada soal ini, Anda diminta menaikkan level dengan membuat **owner** sebagai **object class**.

```
public class Owner {
   private final String id;
   private final String name;
   private final String email;
   // constructor,getter setter
}
```

   Kemudian tambahkan fungsi setOwner(Owner owner) pada wallet class. Buatkan pengujian untuk method tersebut. Gunakan *methodSource* dan *argument provider* untuk membuat parameter object **owner.**

-